DP-LMI package
User Manual

Version v0.1.0

July 2026

Abstract

This manual describes the current public behavior of the DP-LMI package. The present implementation provides `dpbase` for cell-local Bernstein coefficient storage, `dpmat` for known finite real matrix data on tensor grids, and an initial `dpvar` layer for continuous degree-1 YALMIP-backed decision expressions. Solver-facing `dplmi` assembly, SDP solver invocation, and relaxation-lemma workflows remain future package layers in this repository version.

Contents

1

Installation And Verification

1.1 Add The Package To The MATLAB Path

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

```
projectRoot = "path/to/Differential Parameter–Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local development from the repository root, the shorter form is enough:

```
addpath(pwd)
```

1.2 Run The Current Test Suite

The current verification entry point runs the non-SDP tests for `+helper`, `@dpbase`, `@dpmat`, and `@dpvar`. The `@dpvar` tests require YALMIP to be available on the MATLAB path.

```
results = tests.run_all;
table(results)
```

Expected result: all currently implemented tests pass. The suite covers helper validation, `dpbase` construction, grid metadata, local storage, label ordering, rate-bound validation, and current `dpmat` construction, evaluation, algebra, display, plotting, matrix operations, indexing, assignment, structural transforms, and error boundaries. It also covers the current `dpvar` constructor, continuity-by-shared-coefficients behavior, affine algebra, mixed known/decision products, matrix operations, indexing, assignment, and validation boundaries.

The command above does not verify `dplmi` assembly, SDP solver calls, relaxation lemma assembly, or DP-LMI feasibility workflows because those public layers are not implemented in this version.

2

Quick Start

`dpbase` stores one flat Bernstein coefficient cell for each physical cell of a tensor grid. `dpmat` builds on that storage to represent known parameter-dependent matrices.

2.1 A Scalar Grid

```
obj = dpbase({[0 1 3]}, [2 3], 1);

obj.MatrixSize
obj.Degree
obj.GridInfo.Bounds
obj.npar()
obj.ncell()
obj.ncoeff()
size(obj)
obj.cells()
obj.lbls()
```

Expected structural outputs:

```
MatrixSize =
     2     3

Degree =
     1

Bounds =
     0     3

npar()   -> 1
ncell()  -> 2
ncoeff() -> 2
size(obj)-> [2 3]

cells() =
     1
     2

lbls() =
     0
     1
```

There are two physical intervals, $[0, 1]$ and $[1, 3]$. Because the degree is one and the grid is one-dimensional, each physical cell stores two local Bernstein coefficients.

2.2 Access Local Coefficients

```
c1 = obj.coeffs(1);
c2 = obj.coeffs(2);
```

Expected result: both `c1` and `c2` are `1x2` cell arrays. Each entry is a numeric zero matrix of size `2x3`. The first cell corresponds to local labels 0 and 1 on the interval $[0, 1]$; the second cell uses the same local labels on the interval $[1, 3]$.

2.3 A Tensor Grid

```
obj = dpbase({[0 1 2], [10 20 30]}, [1 1], 1);

obj.GridInfo.Points
obj.cells()
obj.lbls()
obj.coeffs([2 1])
```

Expected structural outputs:

```
GridInfo.Points =
     0    10
     0    20
     0    30
     1    10
     1    20
     1    30
     2    10
     2    20
     2    30

cells() =
     1     1
     1     2
     2     1
     2     2

lbls() =
     0     0
     0     1
     1     0
     1     1
```

The first parameter index varies more slowly than the second. This row order is used consistently for physical cells and local Bernstein labels.

2.4 A Known Matrix

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);

A.MatrixSize
A.Degree
A.SourceSummary
A.evaluate(0.25)
```

```
C = A + 5;
C.coeffs(1)
```

Expected structural outputs:

```
MatrixSize =
     1     1

Degree =
     1

SourceSummary =
    "coefficient-backed"

A.evaluate(0.25) -> 2.5
C.coeffs(1)      -> {7, 9}
```

The expression `A + 5` returns another coefficient-backed `dpmat` on the same grid.

2.5 A Function-Backed Known Matrix

```
F = dpmat({[0 pi]}, @(rho) sin(rho));

F.SourceSummary
F.evaluate(pi/2)
F.coeffs(1)
```

Expected outputs:

```
SourceSummary =
    "function"

F.evaluate(pi/2) -> 1
F.coeffs(1)      -> {0, 0}
```

Function-backed `dpmat` objects without an explicit `Degree` keep the exact function handle for evaluation. The inherited local coefficients are placeholder zeros used only to satisfy shared `dpbase` storage; they are not Bernstein evidence and cannot enter coefficient algebra.

2.6 A Continuous Decision Variable

```
P = dpvar(2, {[0 1 2]}, "symmetric");

P.MatrixSize
P.Degree
P.IsContinuous
P.ContainsDecision
left = P.coeffs(1);
right = P.coeffs(2);
isequal(getvariables(left{2}), getvariables(right{1}))
```

Expected structural outputs:

```
MatrixSize =
     2     2

Degree =
     1

IsContinuous    -> true
ContainsDecision-> true
shared boundary -> true
```

`dpvar` creates YALMIP decision coefficients on the global grid nodes and reuses the same coefficient handles in adjacent physical cells. Continuity is therefore built into storage; no equality LMI is added by the constructor.

3

Bernstein And Grid Background

3.1 One Cell, Local Coordinates

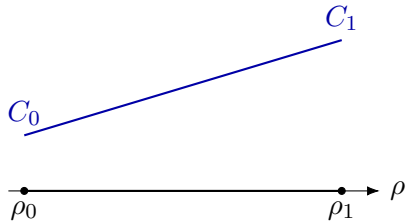
On one physical interval $[\rho_0, \rho_1]$, the package uses the normalized coordinate

$$\alpha = \frac{\rho_1 - \rho}{\rho_1 - \rho_0}, \qquad 0 \le \alpha \le 1.$$

A degree-m Bernstein polynomial is written as

$$P(\alpha) = \sum_{i=0}^{m} C_i B_i^m(\alpha), \qquad B_i^m(\alpha) = \binom{m}{i} \alpha^{m-i}(1-\alpha)^i.$$

For matrix-valued data, each coefficient C_i is a matrix with the same payload size. `dpbase` stores those matrices as one flat coefficient cell for each physical grid cell.



one physical cell, two degree-1 local labels

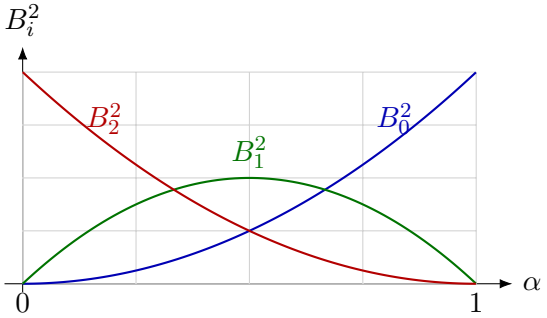
Degree one is the line-segment case. It is useful intuition for piecewise-linear `dpmat` data and for the current degree-1 `dpvar` constructor, but Bernstein storage is not limited to splines.

3.2 Quadratic And Higher-Degree Bases

For degree two, one cell has three local labels $0, 1, 2$:

$$P(\alpha) = C_0\alpha^2 + 2C_1\alpha(1-\alpha) + C_2(1-\alpha)^2.$$

The middle coefficient C_1 controls the interior Bernstein basis function, not an extra physical grid node. The physical interval still has two endpoints; the polynomial degree controls how many local coefficient labels live inside that cell.

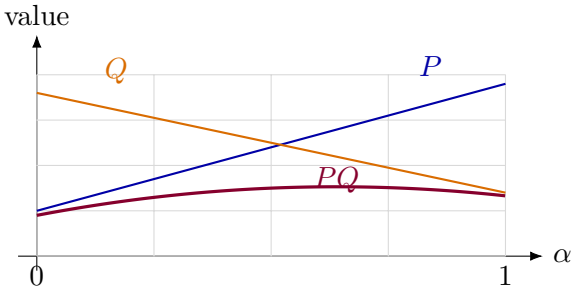


The same pattern extends to any degree m. A cubic cell stores four coefficient labels $0, 1, 2, 3$, a quartic cell stores five, and so on. The labels are local to the current physical cell; adjacent cells have their own local coefficient cells even when continuous objects share boundary handles.

3.3 Products And Degree Growth

Multiplication is the main reason the package treats Bernstein labels as coefficient data rather than as samples. On one cell, multiplying two degree-1 factors produces a degree-2 polynomial:

$$\begin{aligned} P(\alpha) &= \alpha P_0 + (1-\alpha)P_1, \\ Q(\alpha) &= \alpha Q_0 + (1-\alpha)Q_1, \\ P(\alpha)Q(\alpha) &= B_0^2 P_0 Q_0 + B_1^2 \frac{P_0 Q_1 + P_1 Q_0}{2} + B_2^2 P_1 Q_1. \end{aligned}$$



For general scalar degrees n and m,

$$P(\alpha) = \sum_i P_i B_i^n(\alpha), \qquad Q(\alpha) = \sum_j Q_j B_j^m(\alpha),$$

and

$$P(\alpha)Q(\alpha) = \sum_{k=0}^{n+m} R_k B_k^{n+m}(\alpha),$$

where

$$R_k = \sum_{i+j=k} P_i Q_j \frac{\binom{n}{i}\binom{m}{j}}{\binom{n+m}{k}}.$$

For example, if P has degree two and Q has degree three, then the degree-five coefficient with label $k = 2$ is

$$R_2 = \frac{3}{10} P_0 Q_2 + \frac{6}{10} P_1 Q_1 + \frac{1}{10} P_2 Q_0.$$

This is the binomial-normalized convolution used by coefficient-backed `dpmat` multiplication and by mixed known/decision products that remain affine in YALMIP variables.

3.4 Tensor-Product Labels

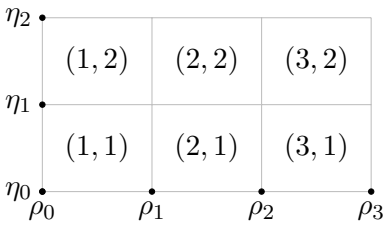
For ℓ parameter dimensions and scalar degree m, each local label is an ℓ-tuple in $\{0, \dots, m\}^\ell$. The number of local coefficients per physical cell is

$$(m+1)^\ell.$$

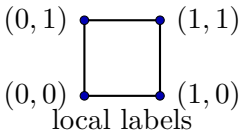
For example, $\ell = 2$ and $m = 1$ gives labels

$$(0,0),\ (0,1),\ (1,0),\ (1,1),$$

which is exactly the order returned by `obj.lbls()`.



physical tensor-grid cells



local labels
inside one cell

Tensor-product multiplication adds labels component-wise. In two dimensions, a label (i_1, i_2) from degree n and a label (j_1, j_2) from degree m contribute to result label

$$(k_1, k_2) = (i_1 + j_1,\ i_2 + j_2),$$

with a scale factor that is the product of the one-dimensional binomial ratios. For three parameters, the same rule applies to triples such as $(i_1, i_2, i_3) + (j_1, j_2, j_3)$. This is why a degree-1 object over a three-dimensional parameter grid has $2^3 = 8$ local coefficient labels per physical cell.

3.5 Data Layout Seen From MATLAB

The physical grid and the local Bernstein labels are separate. A two-parameter `dpmat` with a degree-1 global coefficient grid stores four local coefficients per physical cell:

```
A = dpmat({[0 1], [10 20]}, {1, 3; 5, 7}, Degree=1);

A.npar()
A.ncell()
A.ncoeff()
A.lbls()
A.coeffs([1 1])
A.evaluate([0.25, 12.5])
```

Expected structural outputs:

```
npar()   -> 2
ncell()  -> 1
ncoeff() -> 4

lbls() =
     0     0
     0     1
     1     0
     1     1

coeffs([1 1])-> {1, 3, 5, 7}
evaluate([0.25 12.5]) -> 2.5
```

A degree-2 scalar cell stores three coefficients and evaluates with the quadratic basis shown above.

```
A = dpmat({[0 1]}, {1, 2, 4}, Degree=2);

A.Degree
A.ncoeff()
A.evaluate(0.5)
```

Expected output:

```
Degree   -> 2
ncoeff() -> 3
evaluate(0.5) -> 2.25
```

The current `dpvar` constructor is degree one, but it uses the same cell-local label ordering and tensor-grid storage.

```
P = dpvar(1, {[0 1], [10 20]});

P.Degree
P.ncoeff()
P.lbls()
```

Expected structural output:

```
Degree   -> 1
ncoeff() -> 4

lbls() =
     0     0
     0     1
     1     0
     1     1
```

4

dpbase Reference

4.1 Purpose

`dpbase` is the shared coefficient-backed storage class for gridded DP-LMI objects. It stores tensor-grid metadata, matrix payload size, Bernstein degree, nested physical-cell coefficient data, and rate-dependence metadata.

4.2 Syntax

```
obj = dpbase(gridVectors, matrixSize, degree)
obj = dpbase(gridVectors, matrixSize, degree, localValues)
obj = dpbase(gridVectors, matrixSize, degree, localValues, Name=Value)
```

4.3 Inputs

Input	Description
`gridVectors`	Nonempty cell array of grid node vectors. Each vector must be finite, real, strictly increasing, numeric, and contain at least two nodes.
`matrixSize`	`1x2` positive integer vector giving the row and column size of every coefficient payload.
`degree`	Nonnegative integer scalar Bernstein degree.
`localValues`	Optional nested cell payload. If omitted or empty, `dpbase` creates zero numeric coefficient matrices for every local coefficient of every physical cell.

4.4 Name-Value Options

Option	Description
`IsContinuous`	Logical metadata flag. `dpbase` records the flag but does not enforce boundary sharing itself. Future subclasses own continuity construction.
`ContainsDecision`	Logical metadata flag for future decision-variable subclasses.
`HasRateDependence`	Logical metadata flag. If true, nonempty `RateBounds` must be supplied.
`RateBounds`	Empty, or an $\ell \times 2$ finite real matrix with lower bounds in column one and upper bounds in column two. The number of rows must match the number of parameter dimensions.
`SourceSummary`	String metadata describing the coefficient source. The default is `"coefficient-backed"`.

4.5 Properties

All public properties have private set access. Users can inspect them but cannot assign to them after construction.

Property	Meaning
`GridInfo`	Struct with fields `Vectors`, `Points`, `Bounds`, and `NumNodes`.
`MatrixSize`	Matrix payload size stored as a `1x2` vector.
`Degree`	Bernstein degree.
`LocalValues`	Nested physical-cell storage. A tensor grid uses nested brace access, for example `LocalValues{i1}` followed by `{i2}` and later dimensions; each leaf stores a flat coefficient cell.
`IsContinuous`	Continuity metadata flag.
`ContainsDecision`	Decision-dependence metadata flag.
`HasRateDependence`	True when rate metadata or rate-affine payloads are present.
`RateBounds`	Empty or an $\ell \times 2$ finite real rate-bound table.
`SourceSummary`	String metadata for the coefficient source.

4.6 Methods

Method	Behavior
`size(obj)`	Return the matrix payload size. `size(obj,dim)` follows ordinary matrix behavior for dimensions beyond two by returning one.
`npar(obj)`	Return the number of parameter dimensions.
`ncell(obj)`	Return the number of physical tensor-grid cells.
`ncoeff(obj)`	Return the number of local Bernstein coefficients per cell.
`cells(obj)`	Enumerate physical-cell subscripts in flat row order.
`lbls(obj)`	Enumerate local Bernstein labels in flat row order.
`coeffs(obj,cellSubs)`	Return the flat coefficient cell for one physical cell. `cellSubs` can be a numeric row vector or a cell array with one physical-cell index per parameter.

4.7 Custom Local Values

For a one-dimensional grid with two physical cells and degree two, each physical cell must store three local coefficients.

```
localValues = {{11, 12, 13}, {21, 22, 23}};
obj = dpbase({[0 1 2]}, [1 1], 2, localValues);

obj.coeffs(2)
obj.coeffs({1})
```

Expected output:

```
obj.coeffs(2)  -> {21, 22, 23}
obj.coeffs({1})-> {11, 12, 13}
```

For a tensor grid, physical cells are nested one parameter dimension at a time. The storage contract is `LocalValues{i1}{i2}...`, not `LocalValues{i1,i2,...}`.

```
mkCoeff = @(offset) {offset+1, offset+2, offset+3, offset+4};
localValues = {
    {mkCoeff(110), mkCoeff(120)}, ...
    {mkCoeff(210), mkCoeff(220)}
    };

obj = dpbase({[0 1 2], [10 20 30]}, [1 1], 1, localValues);
obj.coeffs([2 1])
```

Expected output:

```
ans =
  1x4 cell array
    {[211]}    {[212]}    {[213]}    {[214]}
```

4.8 Rate Metadata

`dpbase` validates rate bounds and rate-affine coefficient payloads, but it does not enumerate rate vertices or assemble LMIs. That solver-facing work is reserved for a future `dplmi` layer.

```
obj = dpbase({[0 1], [10 20]}, [1 1], 0, [], ...
    RateBounds=[-1 1; -2 2]);

obj.HasRateDependence
obj.RateBounds
obj.coeffs([1 1])
```

Expected result: `HasRateDependence` is true, `RateBounds` is the given `2x2` table, and the single local coefficient remains the numeric zero payload.

Rate-affine payloads use a scalar struct with fields `Constant` and `Rate`. The `Rate` field must be a cell array with one matrix per parameter dimension.

```
payload = struct("Constant", eye(2), "Rate", {{ones(2), 2*ones(2)}});
localValues = {{{payload}}};
obj = dpbase({[0 1], [10 20]}, [2 2], 0, localValues, ...
    RateBounds=[-1 1; -2 2]);
```

The stored coefficient represents the compact form

$$C(\dot{\rho}) = C_0 + C_1\dot{\rho}_1 + C_2\dot{\rho}_2,$$

but `dpbase` only stores and validates this payload. It does not solve or test any inequality involving $\dot{\rho}$.

4.9 Validation And Error Boundaries

The constructor rejects malformed inputs early and uses domain-specific error identifiers. The following examples are expected to error:

```
dpbase([0 1], [1 1], 0)                % dpbase:InvalidGrid
dpbase({[0 1 1]}, [1 1], 0)            % dpbase:InvalidGridVector
```

```
dpbase({[0 1]}, [1 0], 0)               % dpbase:InvalidMatrixSize
dpbase({[0 1]}, [1 1], -1)               % dpbase:InvalidDegree
dpbase({[0 1]}, [1 1], 0, [], ...
    HasRateDependence=true)              % dpbase:InvalidRateBounds
```

Plain coefficient payloads at the `dpbase` layer must be finite, real, numeric matrices matching `matrixSize`. Symbolic YALMIP payloads are not accepted by `dpbase`; they belong in future subclasses that can define symbolic algebra and solver assembly coherently.

5

dpmat Reference

5.1 Purpose

dpmat represents known finite real matrix data on a parameter grid. It inherits dpbase storage and adds source handling, exact function evaluation, coefficient-backed algebra, matrix structural operations, and two-dimensional block indexing for known data.

5.2 Syntax

```
A = dpmat(gridVectors, source)
A = dpmat(gridVectors, source, Degree=m)
val = A.evaluate(point)
val = evaluate(A, point)
C = A + B
C = A - B
C = -A
C = +A
C = A * B
C = A * M
C = M * A
T = A.'
H = A'
C = [A, B]
C = [A; B]
C = cat(1, A, B)
C = cat(2, A, B)
B = A(rows, cols)
A(rows, cols) = rhs
d = length(A)
d = height(A)
d = width(A)
d = numel(A)
d = ndims(A)
B = squeeze(A)
tf = isequal(A, B)
d = diag(A)
B = diag(A, k)
B = reshape(A, m, n)
B = reshape(A, [], n)
v = vec(A)
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
B = blkdiag(A, M)
disp(A)
display(A)
```

```
h = plot(A)
h = plot(A, dims, SamplesPerCell=n, Name, Value)
```

`source` can be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested `LocalValues` in the `dpbase` contract.

5.3 Source Modes

Mode	Behavior
Function handle, no `Degree`	Stores the exact `FunctionHandle`, probes the lower grid point only to infer matrix size, sets `Degree=1`, and leaves inherited coefficients as placeholder zeros. Function-only objects use `SourceSummary="function"` and cannot enter coefficient algebra.
Function handle with `Degree`	Validates that the handle is representable as local Bernstein data of that degree, populates `LocalValues`, keeps the exact `FunctionHandle`, and uses `SourceSummary="function-bernstein"`.
Global cell grid	Converts overlapping global Bernstein coefficient grids into flat local coefficient cells. The public helper `helper.cell2bern` exposes this same conversion for inspection. For degree m, the global cell array size must be $(n_{\text{cell}}m) + 1$ along each parameter dimension.
Nested `LocalValues`	Passes explicit local coefficients through the `dpbase` nested-cell contract. The degree is inferred from the flat coefficient count $(m+1)^{\ell}$ when it is not supplied explicitly.

All current `dpmat` objects are continuous known data. Their fixed metadata is:

- `IsContinuous=true`
- `ContainsDecision=false`
- `HasRateDependence=false`

Constructor options that try to override those metadata fields are rejected.

5.4 Evaluation

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
A.evaluate(0.75)

B = dpmat({[0 1], [10 20]}, {1, 3; 5, 7}, Degree=1);
B.evaluate([0.25, 12.5])

F = dpmat({[0 pi]}, @(rho) sin(rho));
F.evaluate(pi/2)
```

Expected outputs:

```
A.evaluate(0.75)        -> 3.5
B.evaluate([0.25 12.5]) -> 2.5
F.evaluate(pi/2)        -> 1
```

Coefficient-backed objects evaluate by Bernstein interpolation on the active physical cell. Function-backed objects evaluate through the retained exact handle. Evaluation points must be finite, real, have one entry per parameter dimension, and lie inside the grid bounds.

5.5 Coefficient Algebra

`dpmat` supports coefficient-backed `+`, `-`, unary `-`, and matrix `*`. Numeric scalars and compatible numeric matrices promote to constant `dpmat` operands.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {10, 20, 30}, Degree=2);
C = A + B;

C.Degree
C.coeffs(1)
```

Expected output:

```
C.Degree   -> 2
C.coeffs(1)-> {11, 21.5, 32}
```

Multiplication raises Bernstein degree through binomial-normalized local coefficient convolution.

```
A = dpmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
B = dpmat({[0 1]}, {[5; 6], [7; 8]}, Degree=1);
K = A * B;

K.Degree
size(K)
K.coeffs(1)
```

Expected output:

```
K.Degree   -> 2
size(K)    -> [1 1]
K.coeffs(1)-> {17, 31, 53}
```

Same-bound mixed scalar grids are aligned on a common refinement before coefficient algebra. Mixed physical bounds are rejected. Function-only `dpmat` objects are rejected by coefficient algebra until exact function-handle algebra is designed.

5.6 Matrix Operations And Indexing

`dpmat` supports transpose, conjugate transpose for real data, horizontal and vertical concatenation, `cat` along dimensions 1 and 2, two-dimensional matrix slicing, block assignment, matrix-payload shape inspection, and common structural transforms. Row and column subscripts support `:`, positive integer vectors, full-length logical vectors, and `end`.

```
A = dpmat({[0 1]}, {
    [1 2 3; 4 5 6], ...
    [10 20 30; 40 50 60]
    }, Degree=1);

lastCol = A(:, end);
firstRow = A([true false], :);
```

```
size(lastCol)
lastCol.coeffs(1)
```

Expected output:

```
size(lastCol)     -> [2 1]
lastCol.coeffs(1)-> {[3; 6], [30; 60]}
```

Assignment accepts numeric constants and compatible `dpmat` blocks. It does not support deletion assignment, matrix growth, unsupported dimensions, or coefficient operations on function-only objects.

```
A = dpmat({[0 1]}, {zeros(2), 2*ones(2)}, Degree=1);
A(:, 2) = 5;
A.coeffs(1)
```

Expected output:

```
A.coeffs(1)-> {[0 5; 0 5], [2 5; 2 5]}
```

5.7 Shape And Structural Methods

Shape methods report the size of each matrix payload, not the number of physical cells or coefficients.

```
A = dpmat({[0 1]}, {zeros(2, 3), ones(2, 3)}, Degree=1);

length(A)
height(A)
width(A)
numel(A)
ndims(A)
squeeze(A)
```

Expected output:

```
length(A) -> 3
height(A) -> 2
width(A)  -> 3
numel(A)  -> 6
ndims(A)  -> 2
squeeze(A)-> A
```

Coefficient-backed structural transforms map the same MATLAB operation over every local coefficient payload.

```
A = dpmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);

vec(A).coeffs(1)
diag(A).coeffs(1)
reshape(A, [1 4]).coeffs(1)
tril(A).coeffs(1)
triu(A, 1).coeffs(1)
```

Expected output:

```
vec(A).coeffs(1)        -> {[1; 3; 2; 4], [10; 30; 20; 40]}
diag(A).coeffs(1)       -> {[1; 4], [10; 40]}
reshape(A,[1 4]).coeffs(1) -> {[1 3 2 4], [10 30 20 40]}
tril(A).coeffs(1)       -> {[1 0; 3 4], [10 0; 30 40]}
triu(A,1).coeffs(1)     -> {[0 2; 0 0], [0 20; 0 0]}
```

For vector payloads, `diag(A,k)` constructs an offset diagonal matrix. `reshape` also accepts one inferred empty dimension.

```
A = dpmat({[0 1]}, {[1; 2; 3], [4; 5; 6]}, Degree=1);

diag(A, 1).coeffs(1)
reshape(A, [], 1).coeffs(1)
```

Expected output:

```
diag(A,1).coeffs(1)      -> {[0 1 0 0; 0 0 2 0; 0 0 0 3; 0 0 0 0],
                             [0 4 0 0; 0 0 5 0; 0 0 0 6; 0 0 0 0]}
reshape(A,[],1).coeffs(1)-> {[1; 2; 3], [4; 5; 6]}
```

`blkdiag` accepts coefficient-backed `dpmat` operands and compatible numeric blocks. It aligns same-bound grids on a common refinement and elevates degree as needed.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 0.5 1]}, {10, 20, 30}, Degree=1);
C = blkdiag(A, 5, B);

size(C)
C.GridInfo.Vectors{1}
```

Expected output:

```
size(C)             -> [3 3]
C.GridInfo.Vectors  -> [0 0.5 1]
```

5.8 Display And Plotting

`disp(A)` prints a compact one-line summary. Plain command-window display or `display(A)` prints matrix size, parameter grid metadata, degree, physical-cell count, coefficients per cell, continuity, source, and function-handle status.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
```

Expected output contains:

```
dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed
```

`plot(A)` samples `evaluate` over one or two selected parameter dimensions and labels each matrix entry in the legend. For objects with more than two parameter dimensions, unselected dimensions are fixed at their lower grid bounds.

```
A = dpmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4);
numel(h)
```

Expected result: `numel(h)` is 2, one plotted line for each matrix entry. Legends use LaTeX labels such as $A_{1,1}$ and the ρ-axis label uses LaTeX formatting.

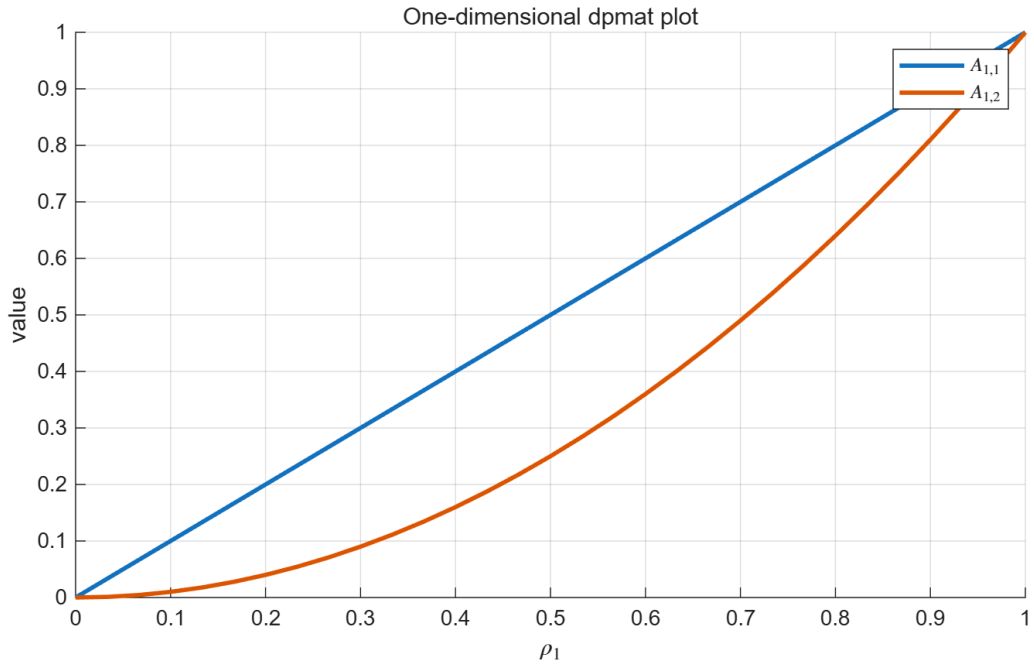


Figure 1: One-dimensional `dpmat` plot generated from MATLAB.

```
A = dpmat({[0 1], [10 20]}, @(rho, eta) [rho + eta, rho - eta]);
h = plot(A, [1 2], SamplesPerCell=2, EdgeColor="none");
numel(h)
```

Expected result: `numel(h)` is 2, one transparent surface for each matrix entry. Two-dimensional plotting defaults to `FaceAlpha=0.5` unless the caller supplies a different value, and accepts ordinary graphics options.

The figure files are generated by running `doc/generate_plot_figures.m` from the repository root. They live under `doc/matlab-figures/` so the manual source, generated plot assets, and regeneration entry point stay in the same documentation tree.

5.9 Validation Boundaries

`dpmat` rejects unsupported source types, malformed global coefficient grids, malformed nested local values, non-finite or complex payloads, bad function arity, empty or complex function outputs, non-polynomial handles when explicit Bernstein evidence is requested, and functions that require a higher `Degree` than the caller supplied. It also rejects constructor attempts to set `IsContinuous`, `ContainsDecision`, `HasRateDependence`, or `RateBounds`.

Operational errors include out-of-bounds or wrong-dimensional evaluation points, function-only coefficient algebra or structural transforms, incompatible matrix sizes, mixed physical grid bounds, `cat` dimensions other than 1 or 2, deletion assignment, matrix-growth assignment, block assignments with incompatible payload size, invalid reshape sizes, invalid diagonal or triangular offsets, and invalid plot dimensions or `SamplesPerCell`.

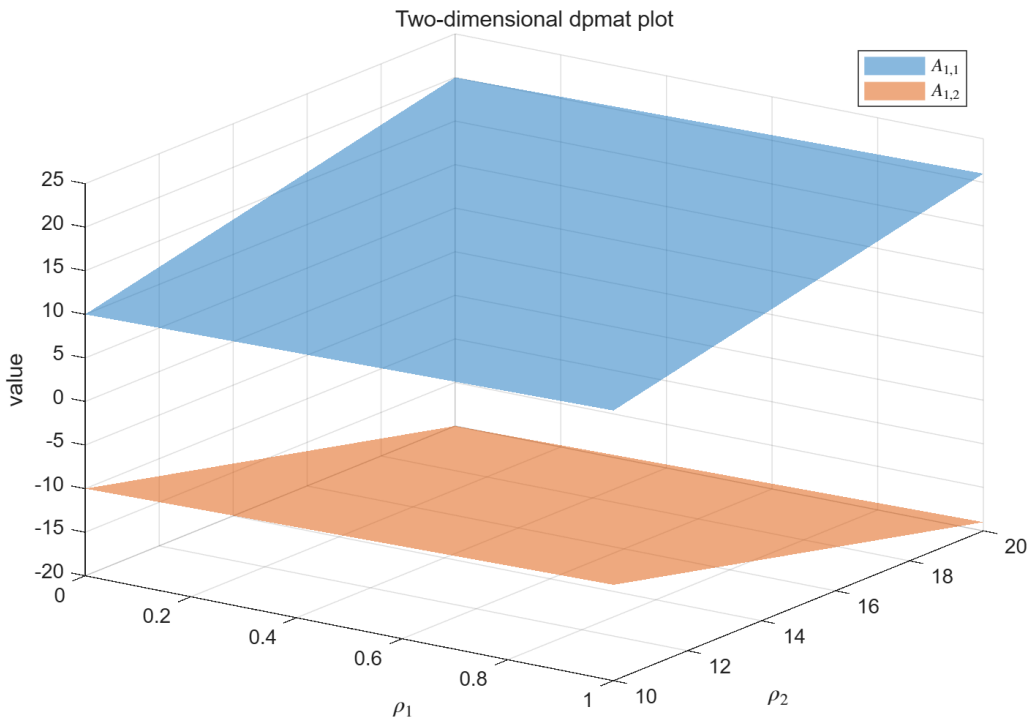


Figure 2: Two-dimensional `dpmat` surface plot generated from MATLAB.

6

dpvar Reference

6.1 Purpose

`dpvar` represents continuous degree-1 Bernstein decision variables on a tensor grid. Its coefficients are YALMIP `sdpvar` matrices stored through the same `dpbase` cell-local layout used by `dpmat`. The current slice is an affine expression layer; it is not an inequality object and does not call an SDP solver.

6.2 Syntax

```
P = dpvar(n, gridVectors)
P = dpvar(n, n, gridVectors)
P = dpvar(n, m, gridVectors)
P = dpvar(..., "full")
P = dpvar(..., "symmetric")
P = dpvar(..., RateBounds=rb)
C = P + Q
C = P - Q
C = -P
C = +P
C = P + A
C = A * P
C = P * A
T = P.'
H = P'
B = P(rows, cols)
P(rows, cols) = rhs
C = [P, A]
C = [P; P]
C = blkdiag(P, 1, A)
D = diff(P)
```

Here `A` may be coefficient-backed `dpmat` data or compatible numeric data. Bare affine `sdpvar` matrices can also be promoted to constant coefficient data in supported affine operations.

6.3 Construction And Continuity

```
P = dpvar(2, {[0 1 2]}, "symmetric");

size(P)
P.Degree
P.SourceSummary
first = P.coeffs(1);
second = P.coeffs(2);
numel(getvariables(first{1}))
isequal(getvariables(first{2}), getvariables(second{1}))
```

Expected structural outputs:

```
size(P)       -> [2 2]
Degree        -> 1
SourceSummary -> "decision"
variables in first endpoint -> 3
shared boundary handle      -> true
```

For a square matrix, `dpvar(n, gridVectors)` follows YALMIP's symmetric default. Use `"full"` when every matrix entry should be an independent decision variable. Rectangular matrices are full by construction.

Adjacent physical cells share the YALMIP coefficient handle at their common boundary. This is how the current constructor preserves continuity; it does not add equality constraints.

6.4 Rate Metadata

`RateBounds` can be supplied as metadata on a `dpvar`. The table has one row per parameter dimension and lower/upper bounds in columns one and two. The constructor stores the metadata outside `LocalValues`; it does not enumerate rate vertices.

```
rb = [−1 2; −3 4];
P = dpvar(1, {[0 1], [10 20]}, RateBounds=rb);

P.HasRateDependence
P.RateBounds
```

Expected output:

```
HasRateDependence -> true
RateBounds        -> [-1 2; -3 4]
```

6.5 Cell-Local Derivatives

`diff(P)` returns a discontinuous cell-local derivative-row `dpvar`. Each physical hypercube stores one coefficient row per parameter direction. The derivative object keeps any `RateBounds` metadata from `P`, but it does not contract those rows with rate vertices or assemble LMIs.

```
P = dpvar(1, {[0 2]}, RateBounds=[−2 3]);
D = diff(P);

D.SourceSummary
D.IsContinuous
D.Degree
D.DerivativeSourceDegree
D.HasRateDependence
rows = D.coeffs(1);
numel(rows)
```

Expected structural output:

```
SourceSummary          -> "derivative"
IsContinuous           -> false
Degree                 -> 0
DerivativeSourceDegree -> 1
```

```
HasRateDependence      -> true
numel(rows)            -> 1
```

For a two-parameter degree-1 object, `diff(P)` stores two derivative rows in each physical cell. Ordinary algebra, indexing, assignment, and concatenation reject derivative-row objects in this slice; row contraction and rate-vertex assembly belong to future solver-facing code.

6.6 Affine Algebra And Products

`dpvar` supports affine coefficient operations and products where at most one side contains decision variables. Products with coefficient-backed known data use the same Bernstein degree-growth rule as `dpmat`.

```
P = dpvar(1, {[0 1]});
A = dpmat({[0 1]}, {10, 20}, Degree=1);

L = A * P;
R = P * A;

L.Degree
R.Degree
L.ContainsDecision
```

Expected output:

```
L.Degree        -> 2
R.Degree        -> 2
ContainsDecision-> true
```

Same-bound mixed scalar grids are aligned on a common refinement. Mixed physical bounds are rejected. Products such as `P * Q`, `P * x` where `x` is a bare decision variable, and products involving rate metadata are rejected in this slice because they would leave the supported affine SDP expression layer.

6.7 Matrix Operations And Indexing

Matrix-like methods report and transform the YALMIP coefficient payloads, not the number of physical cells. The supported structural methods mirror the current `dpmat` coefficient-backed methods.

```
P = dpvar(2, {[0 1]}, "full");

length(P)
height(P)
width(P)
numel(P)
ndims(P)
size(vec(P))
size(diag(P))
size(reshape(P, [1 4]))
size(P(:, end))
```

Expected output:

```
length(P)          -> 2
height(P)          -> 2
width(P)           -> 2
numel(P)           -> 4
ndims(P)           -> 2
size(vec(P))       -> [4 1]
size(diag(P))      -> [2 1]
size(reshape(P,[1 4])) -> [1 4]
size(P(:,end))     -> [2 1]
```

Concatenation, `blkdiag`, structural transforms, matrix slicing, and matrix assignment accept numeric constants, affine `sdpvar` blocks, other compatible `dpvar` objects, and coefficient-backed `dpmat` blocks. They reject nonlinear YALMIP expressions, function-only `dpmat` operands, unsupported dimensions, matrix growth, deletion assignment, and incompatible rate metadata.

6.8 Validation Boundaries

`dpvar` fixes `Degree=1`, `IsContinuous=true`, and `ContainsDecision=true` for the public constructor. Attempts to pass `Degree`, `IsContinuous`, `ContainsDecision`, or `HasRateDependence` as constructor options are rejected. A `"symmetric"` structure flag requires a square matrix. Grid and `RateBounds` validation follows the shared `dpbase` rules.

7

Current Limits

This repository version intentionally has a narrow public surface.

- `dpbase`, `dpmat`, `dpvar`, `helper.chk`, `helper.scanMats`, and `helper.cell2bern` are implemented and covered by the current non-SDP test entry point.
- `dpmat` is a known-data object only. It does not contain YALMIP decisions, rate dependence, exact function-handle algebra beyond evaluation/plot sampling, or solver-facing LMI assembly.
- `dpvar` is currently a continuous degree-1 YALMIP-backed affine expression object with cell-local derivative-row output from `diff(P)`. It does not implement nonlinear decision products, rate-dependent products, derivative-row contraction, inequality assembly, or solver calls.
- `dplmi` remains a future package layer.
- There is no public `dpdiff` class. Future derivative behavior is expected to belong to `dpvar`.
- YALMIP modeling, SDP solver invocation, finite rate-vertex assembly, and relaxation lemma workflows beyond the current affine `dpvar` expression layer are not public behavior yet.
- Private Bernstein utilities support internal tests and implementation, but they are not user-facing functions.

8

Style References Used For This Manual

This manual follows the organization style common in mature MATLAB and control-toolbox documentation: installation first, a small runnable example, conceptual background, then API reference with syntax, arguments, examples, validation rules, and current limits. Local LPVTools, ROLMIP, and ROMULOC documentation were used as style references only. Their APIs and text are not dependencies of this package and are not copied here.